

Companion Proofs:

If It Walks Like an Arbitrage: Protocol-Agnostic Detection with Decidable Structural Equivalence

Anonymous Authors

This document collects the full proofs and the practical structural-equivalence material that accompany the submission. Section 1 contains the mechanized proofs of Theorems 1–5, expanding Appendix D of the paper; Section 2 develops the structural-equivalence query language with worked examples. Cross-references prefixed with `sec:` or `def:` that resolve outside this document refer to the main paper.

1. Full Proofs

We prove the five theorems stated in Section III-F. Throughout, we use the notation of Definitions 1–7 and Algorithms 1–4. We write $|T|$ for the number of nodes in an AST T , and $children(T)$ for the multiset of children of the root of T .

1.1. Proof of Theorem 1 (Preservation)

We first establish a loop invariant that underpins both this theorem and the soundness proof.

Definition 1 (Walk correspondence). *A chain node C in an AST T corresponds to a walk $W = e_1, e_2, \dots, e_m$ in $G = (V, E)$ if each $e_i \in E$ and the address sequence $s(e_1), d(e_1)=s(e_2), \dots, d(e_m)$ is continuous. For a merged node representing parallel paths $C_1, \dots, C_p$, the node corresponds to the set $\{W_1, \dots, W_p\}$ where each W_i is a walk for C_i .*

Proof. We proceed by induction on the number of rewrite rule applications, starting from the initial AST T_0 produced by BUILD-AST.

Base case (T_0). Every Leaf node $\ell = \text{Leaf}(s, d, a, \tau, \sigma)$ in T_0 is constructed by the decode layer from an event in the execution trace (Definition 2). Each such event corresponds to exactly one edge $(s, d, a, \tau, \sigma) \in E$ in the transfer graph (Definition 1). A single edge is a walk of length 1, so the invariant holds for T_0 .

Inductive step. Assume that every leaf and every chain in the current AST satisfies the walk correspondence (leaves trivially, by the base case). We show that each rewrite rule preserves it.

- 1) **Chaining (Algorithm 1, Rule 1).** Let ℓ_1, ℓ_2 be siblings with $d(\ell_1) = s(\ell_2)$. By hypothesis, ℓ_1 corresponds to

a walk $W_1 = e_1, \dots, e_j$ ending at $d(e_j) = d(\ell_1)$, and ℓ_2 to a walk $W_2 = e_{j+1}, \dots, e_k$ starting at $s(e_{j+1}) = s(\ell_2)$. Since $d(\ell_1) = s(\ell_2)$, we have $d(e_j) = s(e_{j+1})$, so the concatenation $W_1 \cdot W_2 = e_1, \dots, e_k$ is a continuous walk in G . The chain node records this concatenation. No edge outside E is introduced: every e_i belongs to W_1 or W_2 , both subsets of E .

The *pool check* ($\sigma \neq d(\ell_1)$) and the *token-mismatch check* ($\tau(\ell_1) \neq \tau(\ell_2)$) are preconditions that restrict when chaining fires, but do not alter the walk: when satisfied, the resulting chain still records the same edges. The same argument applies to the transitive extension (chain + leaf, chain + chain): each operand corresponds to a walk by hypothesis, and the concatenation is valid whenever the junction addresses match.

- 2) **Lifting (Algorithm 1, post-loop).** A fully reduced Tree node $n = \text{Tree}(c, [t_1, \dots, t_m])$ is dissolved: its children $t_1, \dots, t_m$ become children of n 's parent. This operation changes only the tree structure; no chain's internal transfer sequence is modified. Formally, for every chain C that was a descendant of n , the walk W associated with C before lifting is identical to the walk after lifting. Therefore the invariant is preserved.
- 3) **Merging (Algorithm 2, $R_7/R_8/R_9/R_{13}$).** Two chains C_1, C_2 at the same tree level are merged into a single node C_m whenever they share endpoints ($s(C_1) = s(C_2)$, $d(C_1) = d(C_2)$) and satisfy one of the four merge premises: R_7 (parallel paths sharing τ_{out} and differing in τ_{mid}), R_8 (closed cycles with all three tokens matching or *BalCont* at the origin), R_9 (closed cycles sharing τ_{in} and τ_{out}), or R_{13} (R_7 's premises with τ_{in} pinned as well). By hypothesis, C_1 corresponds to walk W_1 and C_2 to walk W_2 . The merged node C_m records the pair $\{W_1, W_2\}$ (Definition 1). Each walk uses only edges from E ; the merge introduces no new edges, regardless of which of the four premise variants fired. The delta map of C_m is the component-wise sum $\delta_m = \delta_1 + \delta_2$: for each address v and token τ , $\delta_m[v, \tau] = \delta_1[v, \tau] + \delta_2[v, \tau]$. This is an arithmetic operation on values already derived from edges in E and introduces no new graph structure.
- 4) **Cycle connection (Algorithm 2, CONNECT-CYCLES).** Two chains C_1, C_2 at the same level with $s(C_1) = s(C_2)$ and complementary token flows ($\tau_{\text{out}}(C_1) = \tau_{\text{in}}(C_2)$) are connected when

their combination satisfies $s(C_1 \cdot C_2) = d(C_1 \cdot C_2)$ (Definition 6). Two sub-cases arise.

Case (a): $d(C_1) = s(C_2)$ (address-adjacent at the junction). By hypothesis, C_1 corresponds to walk W_1 ending at $d(C_1)$ and C_2 to walk W_2 starting at $s(C_2) = d(C_1)$. Their concatenation $W_1 \cdot W_2$ is a valid walk by the same argument as Rule 1, and the closure condition makes it a cycle.

Case (b): both chains are cycles at the same vertex (R8/R9, MERGE-ADD). Both C_1 and C_2 are individually cycles at the shared vertex $v_0 = s(C_1) = d(C_1) = s(C_2) = d(C_2)$, each corresponding to a closed walk W_i in G using only edges from E . The merged node C_m records the concatenation $\text{transfers}(C_1) \cdot \text{transfers}(C_2)$, which forms a multi-walk $\{W_1, W_2\}$ (Definition 1) whose aggregate flow closes at v_0 . Endpoint correspondence holds in the concatenated form: the first leaf of C_m has source v_0 (from W_1) and the last leaf has destination v_0 (from W_2). The delta map $\delta_1 + \delta_2$ captures the aggregate net flow. The closure-under-walk-sets lemma below verifies that this multi-walk representation satisfies Definition 6 as a cycle: each W_i closes individually, and the aggregate closes too. No new edges are introduced.

In both cases, every edge in the cycle node belongs to E .

Closure under walk sets. We verify that the set-of-walks representation used by merged and cycle-connected nodes (Definition 1) preserves the closure property needed by Theorem 3.

Lemma. *If a cycle node C with $s(C) = d(C)$ corresponds to a set of walks $\{W_1, \dots, W_p\}$ per Definition 1, then the union $\bigcup_i W_i$ constitutes a closed flow at $s(C)$: for the traced token τ , the aggregate delta $\delta_C[s(C)] = \sum_i \delta_{C_i}[s(C)]$ equals the net inflow minus outflow at $s(C)$ across all walks.*

Proof. Each walk W_i uses only edges from E (by the inductive argument above). The delta map of the merged node is the component-wise sum $\delta_C = \sum_i \delta_{C_i}$. Since each δ_{C_i} is computed from the edges of W_i alone (amounts in minus amounts out at each address), the sum $\delta_C[s(C)]$ equals the total net inflow minus outflow at $s(C)$ across the union of all walks. The closure condition $s(C) = d(C)$ ensures that $s(C)$ is the common origin of every W_i and that the aggregate flow returns to $s(C)$. The token-match condition $\tau_{\text{in}} =_{\tau} \tau_{\text{out}}$ ensures that the inflow and outflow at $s(C)$ are denominated in equivalent tokens. Therefore $\delta_C[s(C)] > 0$ (verified by VALIDATE-DELTAS) means the aggregate return strictly exceeds the aggregate outlay at $s(C)$, satisfying the profit condition of Definition 7. $\square$

Conclusion. The base case holds for T_0 . Each of the four phases preserves walk correspondence. The closure lemma ensures that set-of-walks nodes satisfy Definition 6 when labeled as arbitrage. By induction on the number of rule

applications, every chain in the final reduced AST corresponds to a walk (or set of walks) using only edges in G , and closed nodes faithfully represent arbitrage cycles. $\square$

Lemma (Linearity). *Each Leaf is consumed by exactly one rule application and removed from its sibling list; no transfer edge appears in more than one chain in the reduced AST.*

Proof. Each rule in Table I consumes its operands and replaces them with a single output node; no rule duplicates a leaf or returns it to a sibling list. By induction on rule applications, leaves are partitioned across the chains of the reduced AST. $\square$

Remark (multi-walk reading of Definition 6). The transfer chain $t_1, \dots, t_k$ in Definition 6 denotes the *concatenated* leaf list of the chain node, which Theorem 1 decomposes into one or more walks in G : a singleton walk for chains built by sequential extension (R6/R10/R12), and a two-element walk-set for chains built by an additive merge (R8/R9). In both forms, the closure condition $s(t_1) = d(t_k)$ refers to the source of the first concatenated leaf and the destination of the last; endpoint correspondence (§1.3, mechanized boundary) ensures these coincide with the chain's structural endpoints $s(C)$ and $d(C)$. In the multi-walk form, each walk W_i is itself closed at the shared vertex v_0 (R8/R9 require both inputs to be cycles), so the aggregate flow returns to v_0 both leaf-wise and at the delta-map level. The token-match condition $\tau(t_1) =_{\tau} \tau(t_k)$ and the positivity condition $\delta_C[s(t_1)] > 0$ of Definition 7 apply to the chain in either form. No extension of Definitions 6 and 7 is needed: they cover single-walk cycles directly and multi-walk cycles via the concatenated leaf list.

Conservation. The statement carries a third clause: T_0 and T_f hold the same *multiset* of transfers, not merely the same set. This is stronger than the inclusion above and is what makes a reported profit non-fabricable. It follows from a single observation about the rules: every conclusion carries exactly the transfers of its premises, concatenated. Chaining and merging replace two operands by one whose transfer list is the concatenation of theirs; lifting relocates subtrees and rewrites no list; annotation changes a label and nothing else. So each step permutes the leaf multiset, and by induction over the reflexive transitive closure so does any reduction sequence (`rcft_transfers_perm` and `rcft_transfers_perm_star` in the mechanization). Two consequences are recorded separately because the paper cites them: no chain in a reachable tree contains a transfer absent from T_0 , counted with multiplicity (`no_fabricated_transfers`), and every balance the system reports at any address and token is a balance of the input trace (`reported_deltas_are_input_deltas`).

1.2. Proof of Theorem 2 (Termination)

Proof. We exhibit a well-founded measure on ASTs and show that each non-trivial pass of ANNOTATE-AND-REDUCE (Algorithm 2) strictly decreases it.

Definitions. For an AST T , let:

- $u(T)$ = number of chain nodes in T not yet annotated (*chaining* or *merging*; chains labeled *cycle*, *arbitrage*, *token burn*, or *token mint* are considered labeled);
- $c(T)$ = total number of immediate children across all Tree nodes in T .

Define $\mu(T) = (u(T), c(T)) \in \mathbb{N} \times \mathbb{N}$, ordered lexicographically. This order is well-founded. It is the measure for the annotation-and-connection loop, matching `measure` in the artifact; the unrestricted relation uses the other order, $(c(T), u(T))$, for the reason given below.

Label monotonicity (Lemma). We first establish that labels are monotonic. A chain receives its label from ANNOTATE-CYCLES: *cycle* when $s(C) = d(C)$, or *arbitrage* when additionally $\tau_{\text{in}} = \tau_{\text{out}}$, $\sigma(C) \notin C \vee s(C) = \sigma(C)$, and $\neg \text{WrapUnwrap}(C)$. Two monotonicity properties hold during the fixpoint loop:

- A labeled chain is never un-labeled. ANNOTATE-CYCLES only adds labels; it never removes them. CONNECT-CYCLES may consume labeled chains by merging them; the merged result is directly assigned a label (*arbitrage* or *cycle*) at merge time, so the number of unlabeled chains does not increase.
- The only label downgrade (*arbitrage* $\rightarrow$ *cycle*) occurs in VALIDATE-DELTAS, which runs *after* the fixpoint has converged and does not affect μ .

Therefore $u(T)$ is non-increasing across passes of ANNOTATE-AND-REDUCE.

Decrease per pass. Algorithm 2 applies two sub-steps to the current tree T :

- 1) $T' \leftarrow \text{ANNOTATE-CYCLES}(T, \text{from})$. Every chain C in T with $s(C) = d(C)$ that is not yet labeled receives a label. If at least one such chain exists: $u(T') < u(T)$. If none exists: $u(T') = u(T)$ and $T' = T$ structurally (annotation does not modify the tree topology, so $c(T') = c(T)$).
- 2) $T'' \leftarrow \text{CONNECT-CYCLES}(T', \text{from}, \text{to})$. This step scans for pairs of chains C_1, C_2 at the same level whose concatenation forms a cycle. Each successful connection replaces two children with one, so $c(T'') \leq c(T') - 1$ per merge. The merge consumes two chains and produces one. Crucially, the cycle-connection function (CONNECT-CYCLES) assigns the merged node a label (*arbitrage* or *cycle*) directly, based on the token-match and middleman conditions at merge time. Therefore: if both operands were labeled, the result is also labeled (u unchanged). CONNECT-CYCLES only selects pairs that are already labeled (*cycle*, *token burn*, or *token mint*), or specific *chaining* pairs in the ETH/WETH wrapping case; it never merges two unannotated chains. In all cases $u(T'') \leq u(T')$.

Case analysis. After both sub-steps, exactly one of three cases holds:

- (i) $u(T') < u(T)$: at least one chain was newly labeled. Then $\mu(T'') \leq (u(T'), c(T'')) < (u(T), c(T)) = \mu(T)$ because the first component decreased.
- (ii) $u(T') = u(T)$ and $c(T'') < c(T')$: no new labels, but at least one merge occurred. Then $\mu(T'') = (u(T), c(T'')) < (u(T), c(T)) = \mu(T)$ because the second component decreased while the first is unchanged.
- (iii) $T'' = T$: neither sub-step changed the tree. The guard in Algorithm 2 returns T , and the recursion terminates.

In cases (i) and (ii), μ strictly decreases.

From lex-decrease to additive bound. By the time the annotation-and-connection loop begins, the structural fixpoint has already assembled the chain nodes; the loop creates no fresh leaf chains. From that point on, both u and c are non-increasing: u by label monotonicity and by merging combining two unannotated chains into one; c by merging, lifting, and cycle-connection only removing or reshaping children without net increase. Hence lex-decrease of (u, c) coincides with a decrease of $u + c$ by at least one per pass, yielding the additive bound below.

Bound. Initially, $u(T_0) \leq n$ (at most one unlabeled chain per leaf, since chains arise from chaining leaves). The total children $c(T_0)$ equals the number of edges in the AST, which is $|T_0| - 1$. A tree with n leaves has at most $n - 1$ internal nodes (each internal node has ≥ 2 children), so $|T_0| \leq 2n - 1$ and $c(T_0) \leq 2n - 2$. Neither component ever increases across passes: u is non-increasing by label monotonicity, and c is non-increasing because merging only removes children. Since each non-trivial pass strictly decreases at least one component, it consumes at least one unit from the finite budget $u + c$. The recursion therefore terminates after at most $u(T_0) + c(T_0) \leq n + (2n - 2) = 3n - 2$ passes. The budget inequality $u(T_0) + c(T_0) \leq 3n - 2$ is mechanized as `termination_bound` in the Rocq artifact, derived from `unlabeled_le_transfers` ($u \leq n$) and `cc_plus2_le_twice_ct` (`handshaking`, $c \leq 2n - 2$); together with the loop's mechanized progress (`fixpoint_step_det_decreases`, the decrease of (u, c) across one deterministic step) it yields the $3n - 2$ pass bound.

What the side condition does, and does not, scope. The $3n - 2$ figure is stated for *fully lifted*, non-empty trees, and it is worth being exact about why. Termination itself needs no such hypothesis, though it needs the other ordering of the pair. Chaining creates an unlabelled chain while removing a child, so $u + c$ is not consumed by every structural rule; the mechanization therefore ranks trees by $(c(T), u(T))$, where chaining, merging and lifting each decrease c and annotation holds c fixed and decreases u . Every rule decreases that measure, so the reduction halts from any input and the relation is well-founded whatever order the rules fire in (`rewrite_step_decreases` and `rewrite_step_wf`, the third clause of the theorem). What full lifting buys is the *arithmetic form* of the budget:

the handshake argument needs every internal node to carry at least two children, and without it a unary cascade of depth d over a single transfer has $c = d$ while $3n - 2 = 1$. Note that trimming does not establish the condition: trimming removes structure carrying no transfer, whereas full lifting demands two children per frame, and a frame holding exactly one transfer carries a transfer. The reduction reaches fully lifted trees by lifting, which is precisely why lifting is a rule and not a preprocessing step.

Practical bound. The bottom-up processing strategy (Section III-B) means that leaf-level chaining completes in a single pass per tree level. The fixpoint in Algorithm 2 then operates on the already-flattened tree, where the number of remaining children is bounded by the number of independent paths. In practice the fixpoint converges in k passes, where k grows with the maximum width (sibling count) rather than the depth of the call hierarchy. $\square$

1.3. Proof of Theorem 3 (Soundness)

Proof. The theorem concludes in the transfer graph, not in the AST: from an *Arbitrage* verdict it produces an address v , a token τ , and a bundle of walks over T_0 's own transfers that together form an arbitrage at v in τ in the sense of Definition 7. Nothing in that conclusion names the rewriting, the normal form, or any label the reduction assigns, which is what keeps the statement from being an inversion of our own vocabulary.

The argument has three steps, and only the second carries content. *Inversion*: reading the verdict backwards is bookkeeping over Algorithm 4. *Bridge*: the chain the verdict points at is backed, edge for edge and with multiplicity, by transfers of T_0 ; this is Theorem 1's conservation clause and it is what converts a statement about our tree into a statement about the transaction. *Transfer*: closure, token match and positivity are then read off the chain and descend to the graph unchanged. We take the three in turn, then record the two auxiliary facts (delta decomposition, and the $=_\tau$ neighbourhood) that the aggregate verdict conditions need.

The inversion step proceeds by contrapositive on each condition: violating any one triggers a verdict-determining reason in Algorithm 4, preventing the *Arbitrage* verdict.

Setup. Algorithm 4 accumulates a reason set R and assigns the verdict by strict priority:

- 1) If $\text{NO_CYCLES} \in R$: return *None*.
- 2) If $\text{LEFTOVERS} \in R$: return *Warning*.
- 3) If $\text{FINAL_NEG} \in R$: return *Warning*.
- 4) If $\text{FINAL_MIXED} \in R$: return *Warning*.
- 5) Otherwise: return *Arbitrage*.

The verdict is *Arbitrage* only when R contains none of NO_CYCLES , LEFTOVERS , FINAL_NEG , or FINAL_MIXED . We verify five conditions; note that all hold simultaneously when the verdict is *Arbitrage*, so we may use any condition's consequence when proving another.

Condition 1: Existence. NO_CYCLES is added to R when the set of arbitrage-labeled cycles $\mathcal{C}$ is empty (first check in

Algorithm 4). Since $\text{NO_CYCLES} \notin R$, we have $\mathcal{C} \neq \emptyset$: at least one chain carries the *arbitrage* label.

Condition 2: Closure ($s(C) = d(C)$). ANNOTATE-CYCLES labels a chain as *cycle* only when $s(C) = d(C)$ (the closure check references Definition 6). Promotion to *arbitrage* preserves this property. CONNECT-CYCLES creates a cycle by concatenating chains $C_1 \cdot C_2$ only when $s(C_1 \cdot C_2) = d(C_1 \cdot C_2)$. Therefore every *arbitrage*-labeled chain satisfies $s(C) = d(C)$.

Condition 3: Token match ($\tau_{\text{in}} =_\tau \tau_{\text{out}}$). The promotion from *cycle* to *arbitrage* requires four conditions: (a) $s(C) = d(C)$ (already established), (b) $\tau_{\text{in}}(C) =_\tau \tau_{\text{out}}(C)$, (c) $\sigma(C) \notin C$ or $s(C) = \sigma(C)$ (the cycle's σ does not appear as an intermediary, or is the cycle origin), and (d) $\neg \text{WrapUnwrap}(C)$ (the cycle is not a pure wrap/unwrap roundtrip). Chains satisfying only (a) retain the *cycle* label and are not counted in the arbitrage set $\mathcal{C}$.

Condition 4: Positive net balance ($a_{\text{out}} > a_{\text{in}}$). This is enforced in two stages:

Stage 1: Gross balance. VALIDATE-DELTAS (Section III-D) runs after the fixpoint converges. For each *arbitrage*-labeled chain C , it inspects $\delta_C[s(C)]$ for the traced token. If $\delta_C[s(C)] \leq 0$, the label is downgraded to *cycle*. Therefore every *arbitrage* chain surviving VALIDATE-DELTAS satisfies $\delta_C[s(C)] > 0$ at the gross level (before gas costs).

Stage 2: Net balance. Algorithm 4 computes the aggregate net balance Δ_{net} by subtracting gas costs (block-builder tip + base-fee burn) from the gross balance. We must show that the per-cycle gross positivity from Stage 1 implies aggregate net positivity.

Lemma (Delta decomposition). When $L = \emptyset$ (no leftovers), the aggregate gross balance decomposes as $\Delta_{\text{gross}} = \sum_{C \in \mathcal{C}} \delta_C[s(C)]$ where $\mathcal{C}$ is the set of arbitrage-labeled cycles and each per-cycle scalar $\delta_C[s(C)]$ comes from Stage 1.

Proof. When $L = \emptyset$, every transfer in the transaction is accounted for by a cycle in $\mathcal{C}$ or by the gas-cost transfers. By Theorem 1, each cycle's edges are original transfers from E , and by the Linearity Lemma above no transfer appears in two distinct cycles. The cycles therefore partition all non-cost transfers, and their deltas sum to the aggregate gross balance. $\square$

By Stage 1, every $\delta_C[s(C)] > 0$. Therefore $\Delta_{\text{gross}} > 0$. The net balance is $\Delta_{\text{net}} = \Delta_{\text{gross}} - \text{gas}$. Two reasons guard the verdict: FINAL_NEG is added when Δ_{net} is negative, and FINAL_MIXED is added when it involves multiple tokens with opposing signs. Since neither reason is in R , every token component of Δ_{net} is strictly positive. Therefore $a_{\text{out}} > a_{\text{in}}$ after all costs.

Condition 5: Value-attribution completeness. LEFTOVERS is added to R when $L \neq \emptyset$ (leftover Leaf nodes not absorbed into any cycle or accounted for by gas costs). Since $\text{LEFTOVERS} \notin R$, we have $L = \emptyset$: every transfer in the transaction is accounted for. This is important because an unaccounted outgoing transfer from $s(C)$ could represent

a hidden cost that invalidates the profit calculation. The absence of leftovers guarantees that the delta map faithfully reflects the net balance of the cycle initiator over the deduced transfers.

Conversely, if leftovers are present ($L \neq \emptyset$), the reason LEFTOVERS is added and the cascade returns *Warning* before reaching the *Arbitrage* verdict. Therefore the *Arbitrage* verdict holds if and only if all five conditions are simultaneously satisfied: cycles exist ($C \neq \emptyset$), no leftovers remain ($L = \emptyset$), the gross delta is positive ($\Delta_{\text{gross}} > 0$), the net balance is positive ($\Delta_{\text{net}} > 0$), and it is not mixed.

Conclusion: the witness in G . Let C be a chain the inversion step exposes. By the bridge, C 's leaves are transfers of T_0 counted with multiplicity, so each is an edge of G . Take $v = s(C)$, $\tau = \tau_{\text{in}}(C)$, and for the bundle take the maximal decomposition of C 's leaf sequence into connected runs. Four facts descend. *Closure*: R_{14} fires only when $s(C) = d(C)$, and the endpoint invariant makes the chain's stored endpoints the source of its first leaf and the destination of its last, so the edge sequence closes at v . *Token match*: R_{14} requires $\tau_{\text{in}}(C) =_{\tau} \tau_{\text{out}}(C)$ on the stored boundary tokens, and the same invariant, read for tokens rather than addresses, makes those the tokens of the first and last edge. *Maximality*: the decomposition is by construction the coarsest one, its size being one more than the number of parallel-merge junctions in C ($\text{walk_decomposition_count}$), so merge-free cycles yield the classical single closed walk and a bundle cannot be trivially fragmented into singletons. *Profit*: δ_C is the fold of the per-transfer balances over C 's leaves, so the R_{16} check that C survived is exactly strict positivity at (v, τ) . Together these are the clauses of Definition 7, so the verdict yields a Definition 7 witness over the transaction's own transfers.

That the definition has discriminating power, rather than being satisfied by anything vaguely circular, is checked mechanically: a two-leg swap that leaves in one asset and returns in another meets every other clause and is rejected on the round-trip condition alone (`plain_swap_not_def5`).

Remark. The converse does not hold: the algorithm may classify a genuine arbitrage as *Warning* when leftovers exist or the balance is mixed. Soundness guarantees the absence of false positives for the *Arbitrage* verdict; it does not guarantee the absence of false negatives.

Lemma (Delta consistency under $=_{\tau}$). *Let C be a cycle with $s(t_1) = d(t_k)$ and $\tau_{\text{in}} =_{\tau} \tau_{\text{out}}$, and let $N(\tau) = \{\tau' : \tau' =_{\tau} \tau\}$ denote the $=_{\tau}$ -neighborhood of τ . The economically correct profit condition at $s(C)$ is $\sum_{\tau' \in N(\tau_{\text{in}})} \delta_C[s(C), \tau'] > 0$. When $|N(\tau_{\text{in}})| > 1$, the per-token check $\delta_C[s(C), \tau] > 0$ for a single τ underapproximates the aggregate.*

Proof. The cycle's net flow at $s(C)$ is the sum of incoming minus outgoing amounts across all transfers in C , per token. Under $=_{\tau}$, tokens in $N(\tau)$ represent the same economic asset; true profit is the sum across the neighborhood. When $\tau_{\text{in}} = \tau_{\text{out}}$ strictly, $N(\tau_{\text{in}}) = \{\tau_{\text{in}}\}$ and the sum reduces

to the per-token check, which is what VALIDATE-DELTAS performs in the mechanized core. When $\tau_{\text{in}} \neq \tau_{\text{out}}$ but $=_{\tau}$ -equivalent (e.g., $\text{ETH} \leftrightarrow \text{WETH}$), the sum is over the two-element neighborhood and is computed by the post-fixpoint promotion step, outside the mechanized core. Since $=_{\tau}$ is not transitive, N is not in general a partition; for the native $\leftrightarrow$ wrapped pairs used in practice, $|N(\tau)| \leq 2$. $\square$

Mechanized boundary. The verdict cascade of Algorithm 4 is captured in the mechanized core as `classify : list reason  $\rightarrow$  verdict`, matching the same strict priority order (NO_CYCLES, LEFTOVERS, FINAL_NEG, FINAL_MIXED, *Arbitrage*). VALIDATE-DELTAS is captured by the predicate `validated_arbitrage`, which requires the chain to carry the *Arbitrage* label, a strictly positive gross delta at its origin for the traced token, and the deployment's cost model `net_positive`. The mechanized core establishes a single boundary statement, `classify_structural_refinement`, that exposes the structural shape of every cycle the algorithm surfaces with verdict *Arbitrage*. *Closure*: $s(C) = d(C)$, propagated from ANNOTATE-CYCLES, which only promotes a chain when its origin matches its destination, through the rewriting trace. *Endpoint correspondence*: the chain's structural origin and destination coincide with the source of its first leaf and the destination of its last leaf, so the syntactic endpoints of C are the actual transfer-graph endpoints of its underlying walk. *Walk decomposition*: the leaves of C decompose into a list of valid walks in G , a singleton walk-set for chains built by sequential extension ($R_6/R_{10}/R_{12}$), and a two-element walk-set for chains incorporating a merge ($R_7/R_8/R_9/R_{13}$) where two parallel paths share endpoints. *Inclusion*: every leaf of C is an edge of the original transfer graph T_0 , so no edge is fabricated by the rewriting. *Positive gross delta*: the balance contribution of C at its origin is strictly positive for the traced token. The first four are inherited from `refinement_of_transfer_graph_cycles` and the walk-correspondence theorem (Theorem 1), both established by induction on the reflexive transitive closure of `rewrite_step` starting from a fresh initial CFT (no chains, only leaves and tree nodes from BUILD-AST). The last is the `validated_arbitrage` premise carried through unchanged. That premise conjoins the gross test of R_{16} with the deployment's cost model `net_positive`, so it is the stronger of the two: it bundles R_{16} with the net-profit gate applied afterwards, and a cycle that passes the first and fails the second is the *attempted* tier rather than an arbitrage. Together they establish the cycle structure of Definition 6 and the structural and gross-profit conditions of Definition 7; the remaining net-delta condition (gross profit exceeds gas costs) is the delta-decomposition argument above. $\square$

1.4. Proof of Theorem 4 (Uniqueness)

Proof. We show that the reduced AST produced by the algorithm is uniquely determined by the input trace under the deterministic kernel scanning the trace order that Property 1

fixes. Property 1 supplies the ordered sibling list; the kernel supplies the rule priority and the choice of redex. The claim is not that the unrestricted rewrite relation is confluent: that relation has genuine critical pairs, and what does hold of it is stated at the end of this proof. We decompose the kernel into its four sub-steps and prove that each admits a unique outcome.

Sibling ordering (from Property 1). Sequential execution within each call frame means that siblings of a Tree node are ordered by trace position. In particular, a swap on an AMM produces two transfer events (one inbound, one outbound) within the same call frame, and they appear as consecutive siblings in the AST.

Step 1: Chaining is deterministic. The chaining procedure (Algorithm 1) processes the sibling list of each Tree node by a greedy scan: for each child c_i , it searches the remaining siblings $c_{i+1}, \dots, c_m$ for the *first* (closest in trace order) compatible partner, under a fixed rule-priority cascade.

Determinism rests on three properties of that procedure, none of which concerns how many transfer events a particular pool contract emits.

First, the sibling list is *ordered*. Property 1 gives the children of every Tree node the order in which the execution produced them, so the phrase “the first compatible partner” denotes a well-defined element of the list rather than an arbitrary one. This is the role Property 1 plays in the proof, and its only role: it supplies the order, not the choice of rule.

Second, the scan direction is fixed. The procedure always advances left to right through that order, so two runs on the same tree examine the same candidates in the same sequence.

Third, the rule priority is fixed. When more than one rule applies to a pair the cascade of Algorithm 1 decides which fires, in a fixed sequence, so the outcome at each step is settled by the tree and the cascade together.

The pair the kernel selects is therefore a function of the tree. The transitive extension, chaining a leaf onto a chain or two chains together, scans the same ordered list in the same direction under the same cascade, and is a function of the tree for the same reason.

What the argument does *not* establish is that the choice is immaterial. A child may have several compatible partners, and a different scan order could pair it differently and so produce a differently shaped tree. Theorem 4 does not need that: it asserts uniqueness of the normal form under the specified kernel, not agreement between kernels. Where two orders do diverge, what survives them is stated at the end of this proof: every rule permutes the transfer multiset, so the delta map is conserved, and reassociation of parallel merges is quotiented by κ .

Step 2: Lifting is deterministic. A Tree node n is lifted (its children promoted to the parent) when all children of n are fully reduced (leaves or chains). This condition is structural: it depends only on the node types in the current tree, not on any ordering choice. Two observations:

- The set of liftable nodes is uniquely determined by the current tree state.
- Lifting two independent subtrees n_1, n_2 (neither is an ancestor of the other) commutes: lifting n_1 then n_2 produces the same tree as lifting n_2 then n_1 , because they operate on disjoint parts of the tree.
- For nested subtrees, the bottom-up strategy lifts inner nodes first, which is the only valid order (a parent cannot be lifted until its children are reduced).

Therefore lifting produces a unique result.

Step 3: Merging and connection are deterministic under the kernel. CONNECT-CYCLES performs two operations on chains at the same tree level: *parallel-path merging* (chains with the same origin and destination are combined) and *cycle connection* (chains with complementary token flows are concatenated to form cycles). We address each.

Parallel-path merging. When chains C_1, C_2 share the same origin and destination, they are merged into C_m . The merged node records the union of walks and the component-wise sum of delta maps. Both operations are associative and commutative:

$$(W_1 \cup W_2) \cup W_3 = W_1 \cup (W_2 \cup W_3)$$

$$(\delta_1 + \delta_2) + \delta_3 = \delta_1 + (\delta_2 + \delta_3)$$

So when k chains share endpoints, the walk set and the delta map of the result do not depend on the pairing order. The *shape* does: the binary tree recording the merges is associated differently under different pairings, and κ is what identifies those reassociations. Under the kernel the ordered scan fixes one pairing.

Cycle connection. Two chains C_1, C_2 with complementary token flows are concatenated when the result closes. Ambiguity could arise if a chain C_1 is compatible with C_2 (for connection) and also with C_3 (for merging). In the kernel no ambiguity arises: the scan of the DSE-ordered children list fixes which pair is considered first, so each pass is a function of the tree. Without that discipline what survives is weaker, and it is what the paper actually relies on. Suppose one pass chains C_1 with C_2 and leaves C_3 separate; a later pass then merges the result with C_3 (or vice versa). Alternatively, if the first pass merges C_1 with C_3 , a later pass chains the merged node with C_2 . Both orders consume the same transfers and yield the same delta map: chaining extends a path by concatenation, merging aggregates parallel paths by union, and neither creates or destroys a transfer (Theorem 1). The two orders need not produce the same tree, and we do not claim that they do; they agree on the transfer multiset and therefore on every balance the verdict reads. Termination guarantees that each ordering reaches a fixpoint (Theorem 2), and the fixpoint guard exits only when no further rule applies.

Step 4: Annotation is deterministic. ANNOTATE-CYCLES labels each chain based on structural properties: closure ($s(C) = d(C)$), token match ($\tau_{\text{in}} =_{\tau} \tau_{\text{out}}$), intermediary membership ($\sigma(C) \notin C$ or $s(C) = \sigma(C)$), and the *WrapUnwrap* guard (R14 only). These are pure predicates

on the chain's attributes, independent of any ordering. $\sigma(C)$ is itself a total function of the trace (the σ cascade selects a unique value when the chain's transfer set admits two valid orientations). Given the same chain, the label is always the same.

Composition of the fixpoint loop. Algorithm 2 applies two sub-steps per pass: first ANNOTATE-CYCLES (Step 4), then CONNECT-CYCLES (Step 3). By Theorem 2, the loop terminates after finitely many passes. We show the sequence of intermediate trees is uniquely determined.

Let T be the input to a pass. ANNOTATE-CYCLES produces T' by labeling chains; this is deterministic (Step 4), so T' is uniquely determined by T . CONNECT-CYCLES produces T'' from T' by merging and connecting chains; this is deterministic under the fixed scan (Step 3), so T'' is uniquely determined by T' . The fixpoint guard checks $T'' = T$; since T'' is unique, the guard's outcome is deterministic.

By induction on the pass number: if T_i is uniquely determined, then T_{i+1} is uniquely determined. The initial tree T_0 (output of leaf manipulation) is unique by Steps 1–2. Therefore the entire sequence $T_0, T_1, \dots, T_k$ is unique.

Full pipeline. The complete algorithm applies:

- 1) BUILD-AST: deterministic (trace $\rightarrow$ AST is a function).
- 2) Leaf manipulation (Algorithm 1): chaining is deterministic (Step 1) and lifting is deterministic (Step 2).
- 3) Node manipulation (Algorithm 2): the fixpoint is unique (composition above).
- 4) VALIDATE-DELTAS: deterministic (pure predicate on delta maps).
- 5) Classification (Algorithm 4): deterministic (priority cascade on reason set).

Each stage produces a unique output from its input. Therefore the final reduced AST and the verdict are uniquely determined by the input execution trace.

Mechanized proof. In the Rocq mechanization, the step is realized as a computable function whose determinism is immediate from the function definition. Uniqueness then follows by induction on the derivation length: if $T_0 \rightarrow^* T_f$ and $T_0 \rightarrow^* T'_f$ with both T_f, T'_f normal forms, determinism forces $T_f = T'_f$. Combined with the constructively-proved well-foundedness of the measure μ , this yields the uniqueness corollary $\exists! T_f. T_0 \rightarrow^* T_f$. $\square$

Remarks. *Critical pairs under the free relation.* Freed from the kernel's order the relation does have genuine critical pairs, and two families matter. A bundle of three mutually mergeable chains can associate as $(C_1 \parallel C_2) \parallel C_3$ or as $C_1 \parallel (C_2 \parallel C_3)$: syntactically distinct trees that no further rewriting reconciles. And a chain can be simultaneously an extension operand (R6, R10, R12) and a parallel-merge operand (R7, R13); firing either rule destroys the other's redex. The first family is resolved by a canonical map: it flattens every nest of merges to its operand list and sorts by trace position, which Property 1 makes a strict order,

so both associations have the same image, and the map is idempotent. On *well-formed* trees, where the transfers carry pairwise-distinct trace positions (linearity, true of every decoded tree by construction and preserved by every rule) and the second family's overlap is excluded by the frame condition, any two one-step reducts of the same tree rejoin up to that map (`local_confluent_mod_kappa_holds`, the theorem's third clause). Whether local joinability extends to a global statement is open, and we say so: the frame condition is not known to be preserved along a reduction, so Newman's lemma does not apply and the assembly is blocked at the induction step rather than at the coherence conditions. Nothing in the paper rests on the outcome; every stated result uses the deterministic order.

What holds unconditionally: conservation. The second clause of the theorem is not a confluence statement at all. Every rule permutes the transfer multiset, and the balance of Definition 3 is a fold over that multiset, so any two trees reachable from T_0 , under any rule orders whatsoever, agree on $\delta[v, \tau]$ at every address and every token (`observable_confluence_delta`). The *balance* component of the verdict is therefore order-independent by conservation, with no well-formedness hypothesis. We do not claim this for the whole verdict: the cascade also reads cycle existence and the leftover list, which are structural rather than delta-determined, so the detector's answer is defined under the deterministic kernel. The canonical map identifies reassociations of parallel merges.

Routers parametricity. Theorem 4 holds for any fixed instantiation of `is_singleton_router`: different Routers sets induce different normal forms, but the convergence and uniqueness theorems are insensitive to the choice.

1.5. Proof of Theorem 5 (Decidability)

Proof. The statement is about the kernel, and it is worth being precise about which relation is meant. By Theorem 2 every term reaches a normal form, and by Theorem 4 the kernel's normal form is unique, so the kernel is convergent and every term has a unique $T\downarrow$. The free relation is *not* claimed convergent here; joinability below is joinability under the kernel.

It remains to show the equivalence. Suppose $T \equiv_R T'$, i.e. T and T' are joinable: $\exists U. T \rightarrow^* U \leftarrow^* T'$. By confluence, U can be further reduced to $T\downarrow$ and to $T'\downarrow$; uniqueness gives $T\downarrow = T'\downarrow$. Conversely, if $T\downarrow = T'\downarrow$ then $T \rightarrow^* T\downarrow \leftarrow^* T'$, so T and T' are joinable.

Since computing $T\downarrow$ terminates and syntactic equality of trees is decidable, the word problem is decidable. $\square$

The coarser equivalence. Syntactic equality of kernel normal forms is finer than one usually wants: it separates two trees that differ only in how a parallel bundle was associated. The canonical map of §1.4 quotients exactly that, and it is computable and idempotent, so comparing images decides equivalence up to reassociation of parallel merges (`struct_equiv_dec`). Its two laws are proved

separately: associativity of the bundle unconditionally, and commutativity under linearity, which is Property 1 in the form the proof needs, namely that operand signatures are distinct. This is the equivalence the query language of §2 uses.

Proposition (Refinement of transfer-graph cycles). *Let G be the transfer graph of a transaction tx (Definition 1). Every chain in $T\downarrow$ labeled arbitrage forms a closed walk in G using only edges of G . The inclusion is strict.*

Proof. By Theorem 1, each chain in $T\downarrow$ corresponds to walks in G using only edges of G . R14 fires only on chains with $s(C) = d(C)$ at the structural level; via endpoint correspondence (§C.3, mechanized boundary), the chain’s structural endpoints coincide with the transfer endpoints $s(t_1)$ and $d(t_k)$, so the corresponding walk is closed. Strict inclusion is witnessed by Appendix F-C (yield harvest): a transaction with eight closed walks in G none of which satisfies R14’s $\tau_{in} =_{\tau} \tau_{out}$ condition. $\square$

1.6. Rocq Trust Base

The development is one self-contained file of 13501 lines: 5 Theorems (the paper’s five; nothing else in the file is declared a theorem), 432 lemmas, 13 corollaries and one example, with no Admitted obligation and no added axiom. Print Assumptions on each of the five theorems lists exactly the parameters below and nothing else. The development declares 11 Parameters. None is used as an axiom about the rewriting system; every Parameter is either an opaque type (with decidable equality) or a boolean predicate case-split in the proofs. The theorems therefore hold for any extraction instantiation.

- `address` : Type and `address_eq_dec`: opaque address type with decidable equality;
- `token` : Type and `token_eq_dec`: opaque token type with decidable equality;
- `token_equiv` : `token -> token -> bool`: case-split in the proofs, so soundness holds for any boolean instantiation. A Prop predicate `is_token_equiv_well_formed` (reflexivity and symmetry; transitivity not required because $=_{\tau}$ is checked only at cycle boundaries) is a deployment well-formedness obligation, not a theorem precondition. Recall scales with the chosen relation; soundness is invariant.
- `is_burn`, `is_mint`, `is_singleton_router`, `is_token_contract`: four boolean predicates case-split in the proofs. `is_token_contract` is used by the `wrap_unwrap` fixpoint guarding R14.
- `net_positive` : `chain_tree -> bool`: the deployer’s cost model. Realizer choices: gross delta exceeds gas plus block-builder payment on Ethereum; plus L1-data cost on Arbitrum; chain-specific basis on BSC. The predicate is case-split in `validate_deltas` and threaded through `validated_arbitrage`, so the five theorems

are parametric in the cost model and transfer to any chain whose realizer is supplied. The attempted-arbitrage population is formally characterized as the structural arbitrage cycles for which `net_positive` returns false (Corollary `attempted_arbitrage_characterization` in `Arbitrage.v`).

- `trace_key` : `transfer -> nat`: the trace-order key realizing Property 1. It supplies the canonical map with a strict order on sibling operands, which is what makes the reassociation quotient well defined. A deployment discharges it with the trace’s own event indices, unique within a transaction.

Two well-formedness conditions on these parameters are *stated but never assumed*, and so are obligations on a deployment rather than hypotheses of any theorem: `is_token_equiv_well_formed` (reflexivity and symmetry of `token_equiv`) and `is_trace_key_wf` (injectivity of `trace_key`). At extraction, the opaque types are instantiated with the concrete OCaml types used by the implementation; `is_burn` and `is_mint` are instantiated via standard event-signature matching, and `is_singleton_router` uses the optional *Routers* address set (§III).

Phase-3 bridge from kernel to rules.. Each computable Phase-3 step (one annotation pass followed by one merge) is a sequence of declarative `rewrite_step` applications. The annotation pass corresponds to a left-to-right cascade of `RS_annotate_arb` / `RS_annotate_cyc` applications over the children list (`annotate_all_fn_to_rewrite_star`). The merge step corresponds to one `RS_merge_*` application that produces a Merging-labeled chain, followed by an `RS_annotate_*` step that relabels it to Arbitrage or Cycle. Composition is closed by `try_merge_children_to_rewrite_star` under the natural Phase-3 invariant that every chain in the children list is labeled — this is the state `annotate_all_fn` establishes before the merger runs. Under that invariant, the bridge gives unconditional `rewrite_star` coverage of every reduced CFT the computable kernel produces.

2. Structural Equivalence in Practice

Theorem 5 states that two CFTs are equivalent iff their normal forms coincide. This appendix develops the practical consequences of this result in three steps: we first build intuition through a concrete example (Section 2.1), then formalize the notion of strategy family via a context-free tree grammar (Section 2.2), and finally sketch a query language over normal forms (Section 2.4).

2.1. Structural Equivalence by Example

We illustrate structural equivalence on two confirmed arbitrages from the evaluation dataset. The two transactions involve entirely different tokens, different pool contracts,

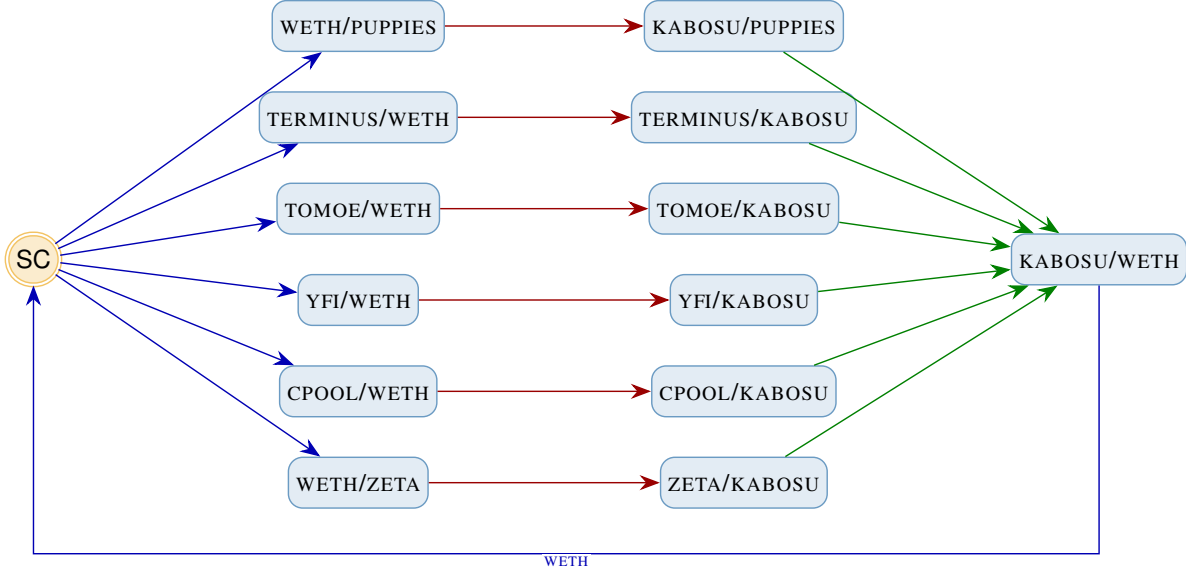


Figure 1: Transfer graph of tx_S . Six parallel paths through twelve pools, same star topology as tx_R (Figure 2).

and a different number of internal swaps, yet their reductions converge to the same topological shape.

tx_R .. The running example from Section III (Figure 2). A smart contract receives 0.0575 WETH and splits it into three parallel swap paths: one through a WETH/MOON pool, one through CHAD/WETH, and one through DUCK/WETH. Each path converts its intermediate token (MOON, CHAD, DUCK) into BEAN at a dedicated second-hop pool. All three BEAN streams converge on a single BEAN/WETH pool, which returns 0.0598 WETH to the smart contract. The raw trace contains 10 transfer events across 6 pool interactions and produces 16 reduction steps.

tx_S .. ¹A different smart contract belongs to the same structural family with more paths (Figure 1). It splits WETH into *six* parallel paths through PUPPIES, TERMINUS, TOMOE, YFI, CPOOL, and ZETA. Each path converts its intermediate token into KABOSU at a dedicated pool, and all six KABOSU streams converge on a single KABOSU/WETH pool. The raw trace contains 19 transfer events across 12 pool interactions.

Reduction (Figure 2)..² Despite their different sizes, both transactions undergo the same reduction sequence. First, leaf chaining (R_1) pairs the inbound and outbound transfers within each pool call frame. After lifting, chain extension (R_6) connects per-pool chains into N multi-hop paths ($N=3$ for tx_R , $N=6$ for tx_S). Then, merge (R_{13}) combines the N paths into a single compound chain: they share origin and destination and agree in both boundary tokens, which is what R_{13} requires over and above R_7 , and they differ in τ_{mid} . A final chaining (R_6) appends the return leg. Annotation (R_{14}) recognizes the closed cycle and labels it as an arbitrage.

The normal forms differ only in arity (3 vs. 6 sub-chains) and in the concrete token names. Abstracting over both, $tx_R \downarrow$ and $tx_S \downarrow$ instantiate the same parametric shape:

$$\text{WETH} \xrightarrow{N \text{ paths}} \tau_i \rightarrow \text{TARGET} \rightarrow \text{WETH}$$

Their exact normal forms differ, so comparing normal forms directly separates the two. What they share is the skeleton: applying sk to both and comparing the results decides their common structural family, and §2.4 casts the two tests as the queries Q_{eq} and Q_{fam} . Theorem 5 is what makes either comparison decidable. We chose a simple example where the shared shape is visually obvious. The value of the algebra is that the same mechanical procedure, reduce and compare, decides equivalence for any pair of words derivable from the rewriting system, including transactions whose raw traces contain hundreds of transfers and nested contract calls where visual assessment is impossible.

Implications..³ The normal form opens several directions beyond detection. A system that indexes reduced CFTs could retrieve *all* star-shaped convergent arbitrages from a historical database in a single structural query, regardless of which tokens or pools were used. Normal forms also provide a natural feature space for unsupervised learning: clustering on normal-form trees groups transactions by fund flow topology, suggesting strategy families without labeled training data. The formal guarantee attaches to exact equality, not to clustering: two transactions whose normal forms are identical are structurally equivalent (Theorem 5), whereas a cluster of nearby normal forms is a heuristic grouping whose members need share no structural property beyond whatever the chosen metric measures.

The equational theory also induces a natural *taxonomy*. At the finest level, two transactions belong to the same equivalence class iff their normal forms are syntactically identical; tx_R ($N=3$) and tx_S ($N=6$) fall in different

1. 0xaa4b2aec9c7a571f96062576135632ece91bde142075963219b26232404dfc5c

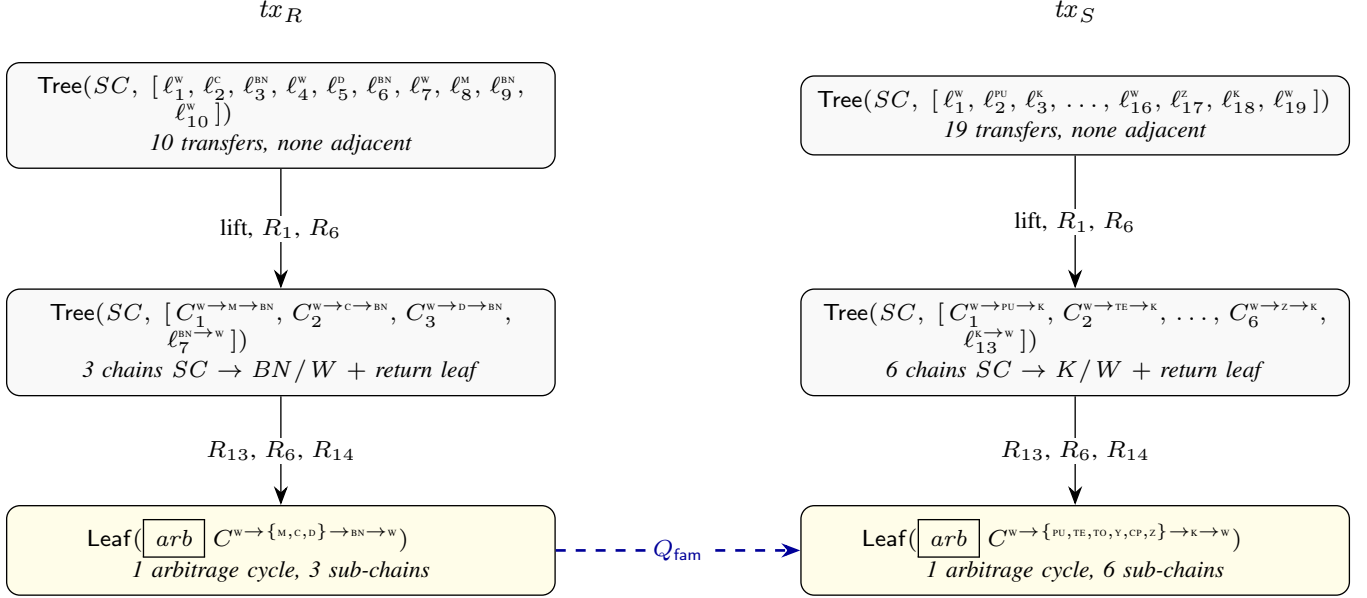


Figure 2: Side-by-side reduction of tx_R and tx_S . Superscripts give each transfer’s token. Lifting dissolves the call frames that separate the transfers, R_1 pairs the first two of each arm and R_6 extends it with the third; R_{13} merges the arms, which share both endpoints and both boundary tokens and differ only in τ_{mid} ; R_6 closes the aggregate with the return leaf and R_{14} annotates. Despite different tokens and arities (3 vs. 6), both reduce to a single arbitration cycle with N merged sub-chains. The dashed arrow marks family equivalence (Q_{fam} , Section 2.4): different normal forms, same topology.

classes. At a coarser level, one can quotient over arity and token identities to obtain *parametric families* where both belong to the same family (star-convergent arbitrage with variable N). The following sections formalize this hierarchy.

2.2. A Context-Free Grammar for Strategy Families

The parametric shape shared by tx_R and tx_S can be described by a family of context-free grammars. We introduce two constructors over normal forms: $\text{Star}(\alpha, L, \alpha)$ denotes an arbitrage-annotated cycle (R14) with initiator token α , and $\text{Swap}(\tau, \tau')$ denotes a transfer chain with entry token τ and exit token τ' . We write $\circ$ for sequential chaining (within an arm) and $\parallel$ for parallel composition (across arms). Let Σ be a countably infinite set of token identifiers. For each pair $(\alpha, \tau') \in \Sigma^2$, define the grammar $G_{\alpha, \tau'} = (\{S, L, C, H\}, \Sigma, P_{\alpha, \tau'}, S)$ where the production set $P_{\alpha, \tau'}$ is:

$$\begin{aligned}
 S &\rightarrow \text{Star}(\alpha, L, \alpha) \\
 L &\rightarrow C \parallel L \mid C \\
 C &\rightarrow \text{Swap}(\alpha, \tau_1) \circ H_{\tau_1} \\
 H_{\tau} &\rightarrow \text{Swap}(\tau, \tau') \mid \text{Swap}(\tau, \tau_k) \circ H_{\tau_k} \quad \text{for any } \tau_1, \tau_k \in \Sigma
 \end{aligned}$$

The *star-convergent family* is the language $\mathcal{L}_{\text{star}} = \bigcup L(G_{\alpha, \tau'})$ ranging over $(\alpha, \tau') \in \Sigma^2$. Each individual grammar $G_{\alpha, \tau'}$ is context-free: the start symbol determines α (the initiator token), the production for C binds the

convergence token τ' across all branches, and the intermediate tokens τ_1, τ_k are freely chosen per cycle. Strictly, H_{τ} is a family of nonterminals indexed by τ ; for any finite transaction, only finitely many are reachable. The recursive production H_{τ} allows swap paths of arbitrary length: a two-hop path uses $H_{\tau} \rightarrow \text{Swap}(\tau, \tau')$, a three-hop path uses $H_{\tau} \rightarrow \text{Swap}(\tau, \tau_k) \circ H_{\tau_k}$ with one further expansion, and so on. A bottom-up ranked tree automaton does not enforce that all siblings share the same τ' , since sibling states are computed independently. Because the nonterminals H_{τ} are indexed by token, what we give is a grammar *schema*: any finite token vocabulary instantiates it to a finite context-free grammar, in which membership is decidable. For finite Σ an unranked hedge automaton could also encode τ' in its state space; the grammar is the more natural and compositional specification either way.

Remark. In practice, Σ is finite: the number of deployed token contracts on any chain is finite, and the gas limit of a transaction bounds the number of transfers (and hence distinct tokens) in any single normal form. The arity N of a derivation is similarly bounded. The grammar is defined over a countable alphabet for generality, but every concrete word has bounded depth (Theorem 2) and bounded width (gas). The grammar is extensible: new strategy families (e.g., flash-loan-wrapped arbitrages, sequential compositions) require only new productions in S , not changes to the rewriting system.

Recall the skeleton map sk (§III-G) is an α -abstraction; the grammar $G_{\alpha, \tau'}$ enumerates its orbits parametrically. The rewriting rules R1–R15 inspect only the structural predi-

cates (is_burn, is_mint, is_singleton_router, $=_\tau$), so for any renaming π preserving them, $\text{sk}(\pi(T\downarrow)) = \pi(\text{sk}(T\downarrow))$. The Rocq mechanization makes this parametricity explicit: address and token are declared as opaque types.

Proposition 1 (Grammar characterization). *Let $T\downarrow$ be a normal form containing an arbitrage cycle (Definition 7) with hub address h , entry/exit token α , and one or more parallel swap branches converging at a common intermediate token τ' . Then $T\downarrow \in L(G_{\alpha,\tau'})$. Conversely, every word in $L(G_{\alpha,\tau'})$ is the structural image of such a star-convergent arbitrage.*

Sketch. ($\Rightarrow$) By termination and confluence (Theorems 2 and 4), the chaining and lifting rules (R1, R6) collapse each branch into a swap sequence $\text{Swap}(\alpha, \tau_1) \circ \dots \circ \text{Swap}(\tau_{k-1}, \tau')$, matching $C \rightarrow \text{Swap}(\alpha, \tau_1) \circ H_{\tau_1}$ with H unfolded as needed (finite by termination). Parallel sibling branches at the hub compose via $L \rightarrow C \parallel L$. The outer $\text{Star}(\alpha, L, \alpha)$ records that branches return to hub h 's account in token α , the closure witnessed by ANNOTATE-CYCLES (R14). ($\Leftarrow$) Any derivation in $G_{\alpha,\tau'}$ exhibits a Star node whose branches route tokens through τ' and return to α ; this is a valid CFT normal form by construction, and R14 annotates it as an arbitrage when $\sigma(C) \notin C \vee s(C) = \sigma(C)$ and $\neg \text{WrapUnwrap}(C)$, with $\delta > 0$ checked externally by VALIDATE-DELTAS . $\square$

The normal form of tx_R ($N=3$) is:

$$\begin{aligned} tx_R\downarrow = & \text{Star}(\text{weth}, (\text{Swap}(\text{weth}, \text{moon}) \circ \text{Swap}(\text{moon}, \text{bean})) \\ & \parallel (\text{Swap}(\text{weth}, \text{chad}) \circ \text{Swap}(\text{chad}, \text{bean})) \\ & \parallel (\text{Swap}(\text{weth}, \text{duck}) \circ \text{Swap}(\text{duck}, \text{bean})), \text{weth}) \end{aligned}$$

The normal form of tx_S ($N=6$) is:

$$\begin{aligned} tx_S\downarrow = & \text{Star}(\text{weth}, (\text{Swap}(\text{weth}, \text{puppies}) \circ \text{Swap}(\text{puppies}, \text{kabosu})) \\ & \parallel (\text{Swap}(\text{weth}, \text{terminus}) \circ \text{Swap}(\text{terminus}, \text{kabosu})) \\ & \parallel (\text{Swap}(\text{weth}, \text{tomoe}) \circ \text{Swap}(\text{tomoe}, \text{kabosu})) \\ & \parallel (\text{Swap}(\text{weth}, \text{yfi}) \circ \text{Swap}(\text{yfi}, \text{kabosu})) \\ & \parallel (\text{Swap}(\text{weth}, \text{cpool}) \circ \text{Swap}(\text{cpool}, \text{kabosu})) \\ & \parallel (\text{Swap}(\text{weth}, \text{zeta}) \circ \text{Swap}(\text{zeta}, \text{kabosu})), \text{weth}) \end{aligned}$$

Both derivations belong to $\mathcal{L}_{\text{star}}$: $tx_R\downarrow \in L(G_{\text{weth}, \text{bean}})$ with $N=3$, and $tx_S\downarrow \in L(G_{\text{weth}, \text{kabosu}})$ with $N=6$. Any star-convergent arbitrage belongs to $\mathcal{L}_{\text{star}}$ for the appropriate (α, τ') : a 10-arm star through ten different memecoins, a 2-arm path through stablecoins, or a star with three-hop swap chains through nested intermediaries. The grammar generates an infinite family; tx_R and tx_S are two concrete members.

Structural equivalence at the exact level ($\equiv_R$) distinguishes them because their normal forms differ in arity and token names. Family equivalence (Q_{fam} in Section 2.4) identifies them: any two words in $\mathcal{L}_{\text{star}}$ belong to the same strategy family, regardless of how many arms they have or which tokens they route through.

2.3. Extending the Grammar: Flash-Loan Wrappers

The grammar is extensible without modifying the rewriting system. We illustrate this by adding productions that capture flash-loan-wrapped arbitrages, the family that contains 63% of our exclusive confirmed detections (Category 2, §IV-C). The extension is instructive because it forces us to reason about what survives the fixpoint.

What the rewriting preserves.. A flash loan consists of two transfers: the borrow $\ell_b = (\text{pool}, \text{bot}, a, \tau_\ell)$ and the repay $\ell_r = (\text{bot}, \text{pool}, a, \tau_\ell)$, with identical amount, identical token, and reversed endpoints. These form a specular pair in the sense of §III-D. The leftover recovery pass removes the pair from the leaf list and emits a *leftover cycle*, a first-class annotated node that records the loan token and amount. Leftover cycles are preserved by every subsequent rewrite rule (they are neither chained, lifted, nor merged with ordinary chains), so they survive into the normal form. The arbitrage cycle that *uses* the borrowed capital reduces independently under the standard star-convergent rules; the two appear side by side in the normal form.

Grammar extension.. A normal form is a pair (N, Λ) where N is a star-convergent derivation and Λ is a (possibly empty) sequence of leftover cycles whose token coincides with the star's hub. For each $(\alpha, \tau') \in \Sigma^2$, define $G_{\alpha,\tau'}^F = (\{F, N, L, C, H, \Lambda\}, \Sigma, P_{\alpha,\tau'}^F, F)$ with productions:

$$\begin{aligned} F & \rightarrow (N, \Lambda) \\ N & \rightarrow \text{Star}(\alpha, L, \alpha) \\ L & \rightarrow C \parallel L \mid C \\ C & \rightarrow \text{Swap}(\alpha, \tau_1) \circ H_{\tau_1} \\ H_\tau & \rightarrow \text{Swap}(\tau, \tau') \mid \text{Swap}(\tau, \tau_k) \circ H_{\tau_k} \\ \Lambda & \rightarrow \varepsilon \mid \text{LC}(\alpha, a) \cdot \Lambda \end{aligned}$$

As in §2.2, H_τ is a family of nonterminals indexed by τ . When $\Lambda = \varepsilon$, $G_{\alpha,\tau'}^F$ reduces to the star grammar $G_{\alpha,\tau'}$; when Λ is non-empty, the LC tokens coincide with the hub α , capturing flash-loan wrappers on the cycle's entry token. The extension thus subsumes pure stars and adds flash-loan-wrapped arbitrages without modifying the rewriting system. The productions generate all (possibly empty) sequences of $\text{LC}(\alpha, \cdot)$; we define $L(G_{\alpha,\tau'}^F)$ as the image after canonicalizing Λ to its sorted representative (ascending amount). Over the practical finite alphabet of amounts, $L(G_{\alpha,\tau'}^F)$ is context-free and membership is decidable in linear time; queries below treat Λ as a set ($\text{LC} \in \Lambda$). The total star-extended family is

$$\mathcal{L}_{\text{arb}} = \bigcup_{(\alpha, \tau') \in \Sigma^2} L(G_{\alpha,\tau'}^F),$$

and the flash-loan family $\mathcal{L}_{\text{flash}} \subset \mathcal{L}_{\text{arb}}$ is the subset with Λ non-empty. Specular pairs on tokens unrelated to the cycle (a refund, a meta-transaction relay) do not satisfy any $G_{\alpha,\tau'}^F$ and are therefore outside the family. Membership is decidable in linear time in the size of the normal form.

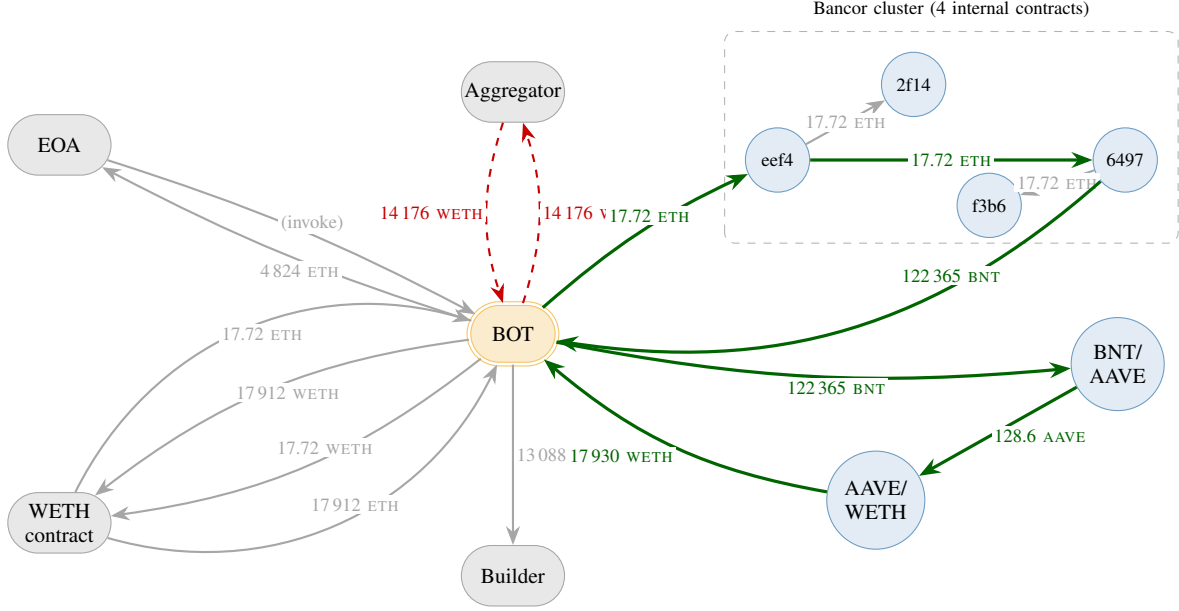


Figure 3: Full transfer graph of the 17912 WETH arbitrage, all 14 transfers in the decoded trace. Bancor comprises four internal contracts (labels show the first four hex characters of each address) with three internal ETH hops that the algorithm routes through. **Green**: the star-convergent cycle our system extracts ($\text{Star}(\text{weth}, S_{\text{BAW}}, \text{weth})$). **Dashed red**: the flash-loan specular pair, collapsed by leftover recovery into $\text{LC}(\text{weth}, 14176)$. Grey: the remaining transfers (EOA invocation, two wrap/unwrap pairs, builder bribe, profit return). The narrative in §F-B flattens the Bancor cluster into a single node.

The 17912 WETH arbitrage as a concrete member.. We illustrate on Ethereum transaction `0x45388b0f9ff46ffe98a3124c22ab1db2b1764ecb3b61234e29e5c9732b7fd4ab`, a confirmed arbitrage from the exclusive detections that Eigenphi does not flag (full forensic analysis in Appendix F-B). A bot borrows 14175.71 WETH from an aggregator, executes a three-hop cycle through Bancor, a BNT/AAVE pool, and an AAVE/WETH pool, and repays the loan.

Figure 3 shows the full decoded transfer graph, with the detected star-convergent cycle highlighted in green and the flash-loan specular pair in dashed red; remaining transfers (wrap/unwrap, builder bribe, EOA invocation) are shown in grey.

After reduction, the main arbitrage cycle annotates under R14; the borrow/repay pair produces a leftover cycle under the specular rule; the profit unwrap is a second annotated cycle on WETH (native/wrapped equivalence). Writing

$$S_{\text{BA}} = \text{Swap}(\text{weth}, \text{bnt}) \circ \text{Swap}(\text{bnt}, \text{aave})$$

for the arm and ignoring the profit unwrap (a separate star on WETH that contributes no additional content to the family-membership test), the relevant normal-form pair is

$$(N, \Lambda) = (\text{Star}(\text{weth}, S_{\text{BA}}, \text{weth}), \text{LC}(\text{weth}, 14175.71)),$$

a word in $L(G_{\text{weth}, \text{aave}}^F)$, hence in $\mathcal{L}_{\text{flash}}$. Note where the third swap went: the arm ends at the convergence token $\tau' = \text{aave}$, and the closing $\text{aave} \rightarrow \text{weth}$ swap is the cycle's return,

recorded by the Star rather than by any arm, exactly as $\text{kabosu} \rightarrow \text{weth}$ is for tx_S . Writing the arm with three swaps would set $\tau' = \alpha$, which the grammar of §2.2 excludes: every arm must reach the convergence token *before* the return. Only genuine borrow/repay wrappers on the cycle's entry token qualify; unrelated specular pairs do not.

Decidability lifts directly.. The family-membership query for $\mathcal{L}_{\text{flash}}$ decomposes into (i) parsing $tx \downarrow$ as a (N, Λ) pair, which is a linear scan of the normal form, and (ii) checking the token-match condition, which is a set comparison on the leftover cycles. Both are decidable in polynomial time. Theorem 4 (uniqueness of normal forms) guarantees that the answer is well-defined under the specified deterministic reduction order. The same decomposition handles other specular-pair wrappers (meta-transaction relays, approval refunds) by varying the matching predicate; the grammar skeleton does not change.

2.4. From Detection to Query Language

The convergent deterministic normalization kernel defines more than a detection algorithm: it induces a decidable equational theory over fund flow trees. We sketch how this theory gives rise to a query language for transaction forensics, and illustrate each query on the running examples tx_R and tx_S .

Normal forms as an intermediate representation.. Every transaction t reduces to a unique normal form $t \downarrow$ (Theorem 4) whose shape encodes the fund flow structure: cycles, chains, leftovers, and their nesting. The rewriting system

acts as a compiler from raw execution traces to a canonical intermediate representation. Any *structural* question about a transaction reduces to a predicate over its normal form, and because the representation is unique, equivalent queries on equivalent traces always agree.

Formally, let $\mathcal{T}$ be the set of AST terms and $\mathcal{N} = \{T \downarrow \mid T \in \mathcal{T}\}$ the set of normal forms. A *query* is a decidable predicate $Q : \mathcal{N} \rightarrow \{\top, \perp\}$.

Q_{arb} : arbitrage detection.. $Q_{\text{arb}}(T \downarrow) = \top$ iff $T \downarrow$ contains a cycle C with $\tau_{\text{in}}(C) = \tau_{\text{out}}(C)$ and $\delta_C[s(C)] > 0$. This is the query implemented by our classifier. On tx_R , the normal form contains one arbitrage cycle with three merged sub-chains, matching entry and exit tokens, and positive delta, so $Q_{\text{arb}}(tx_R \downarrow) = \top$.

Q_{eq} : structural equivalence.. $Q_{\text{eq}}(T_1, T_2) = \top$ iff $T_1 \downarrow = T_2 \downarrow$. Decidable by Theorem 5; enables clustering transactions by fund flow shape without prior labels. For tx_R and tx_S , $Q_{\text{eq}} = \perp$: their normal forms share the same star topology but differ in arity ($N=3$ vs. $N=6$) and token names. Two transactions with the same arity, the same number of hops per path, and identical token identities would satisfy $Q_{\text{eq}} = \top$. The coarser family query Q_{fam} (below) captures their shared structure.

Q_{match} : strategy matching.. Given a reference transaction T_{ref} , $Q_{\text{match}}(T) = \top$ iff $T \downarrow = T_{\text{ref}} \downarrow$. With $T_{\text{ref}} = tx_R$, this query flags every transaction whose normal form exhibits the same three-arm star, regardless of the protocols involved.

Q_{anom} : structural anomaly.. Given a set S of known normal forms for a bot address, $Q_{\text{anom}}(T) = \top$ iff $T \downarrow \notin S$. If the bot behind tx_R later executes a transaction with a four-arm star or a sequential chain, the new normal form falls outside S and the query flags a strategy change.

Q_{fam} : family classification.. The queries above compare normal forms by syntactic equality. A coarser equivalence captures the observation that a three-arm star and a five-arm star, or a star with one-hop and two-hop swap paths, are instances of the same *strategy family*. Define the *skeleton map* $\text{sk} : \mathcal{N} \rightarrow \mathcal{S}$, where $\mathcal{S}$ is the set of skeleton terms (topology without arity, path length, or token names), inductively:

Let $T \downarrow = \text{Star}(\alpha, C_1 \parallel \dots \parallel C_N, \alpha)$ be a normal form with N parallel swap-chain arms, where each $C_i = \text{Swap}(\tau, \tau_{i,1}) \circ \dots \circ \text{Swap}(\tau_{i,k_i-1}, \tau')$ is a k_i -hop chain. The skeleton $\text{sk}(T \downarrow) \in \mathcal{S}$ is computed in three steps:

- 1) *Erase path length*. Collapse each k_i -hop arm to a single swap retaining only the boundary tokens (analogous to R_1):

$$C_i = \text{Swap}(\tau, \tau_{i,1}) \circ \dots \circ \text{Swap}(\tau_{i,k_i-1}, \tau') \mapsto \text{Swap}(\tau, \tau')$$

- 2) *Erase arity*. Merge the N collapsed arms into a single representative (analogous to R_7):

$$\text{Swap}(\tau, \tau')^{\parallel N} \mapsto \text{Swap}(\tau, \tau')$$

- 3) *Erase token identities*. Replace all concrete tokens with a wildcard $\star$:

$$\text{Star}(\alpha, \text{Swap}(\tau, \tau'), \alpha) \mapsto \text{Star}(\star, \text{Swap}(\star, \star), \star)$$

All three steps are decidable (finite tree traversals). Token continuity within each arm ($\tau_{i,k-1}$ feeds into $\tau_{i,k}$) is guaranteed by construction: the chaining rules pair transfers whose junction addresses match, and each swap produces exactly one outgoing token that becomes the next swap's input. Step 1 collapses these junctions, retaining only the boundary tokens. After step 1, all N arms are identical ($\text{Swap}(\tau, \tau')$), so step 2 is a no-op. Step 3 erases the remaining names. The skeleton retains only the topology. Then $Q_{\text{fam}}(T_1, T_2) = \top$ iff $\text{sk}(T_1 \downarrow) = \text{sk}(T_2 \downarrow)$.

Example: $Q_{\text{fam}}(tx_R, tx_S) = \top$. We compute sk on both normal forms.

$tx_R \downarrow = \text{Star}(\text{weth}, \underbrace{C_1 \parallel C_2 \parallel C_3}_{3}, \text{weth})$ where each C_i is

a 2-hop chain:

- 1) Erase path length (per arm):
 $C_i \mapsto \text{Swap}(\text{weth}, \text{bean}) \quad \forall i$
- 2) Erase arity:
 $\text{Swap}(\text{weth}, \text{bean})^{\parallel 3} \mapsto \text{Swap}(\text{weth}, \text{bean})$
- 3) Erase tokens:
 $\text{Star}(\text{weth}, \text{Swap}(\text{weth}, \text{bean}), \text{weth}) \mapsto \text{Star}(\star, \text{Swap}(\star, \star), \star)$

$tx_S \downarrow = \text{Star}(\text{weth}, \underbrace{C_1 \parallel \dots \parallel C_6}_{6}, \text{weth})$ where each C_i is

a 2-hop chain:

- 1) Erase path length (per arm):
 $C_i \mapsto \text{Swap}(\text{weth}, \text{kabosu}) \quad \forall i$
- 2) Erase arity:
 $\text{Swap}(\text{weth}, \text{kabosu})^{\parallel 6} \mapsto \text{Swap}(\text{weth}, \text{kabosu})$
- 3) Erase tokens:
 $\text{Star}(\text{weth}, \text{Swap}(\text{weth}, \text{kabosu}), \text{weth}) \mapsto \text{Star}(\star, \text{Swap}(\star, \star), \star)$

Both reduce to $\text{Star}(\star, \text{Swap}(\star, \star), \star)$. Therefore $\text{sk}(tx_R \downarrow) = \text{sk}(tx_S \downarrow)$ and $Q_{\text{fam}}(tx_R, tx_S) = \top$: same family despite different arities ($N=3$ vs. $N=6$), different tokens, and different pools.

A hypothetical five-arm arbitrage through longer paths produces the same skeleton. In contrast, a sequential arbitrage (one cycle feeds into the next) would have a different skeleton topology and belong to a different family. The skeleton quotient thus induces a principled taxonomy of MEV strategies directly from the algebraic structure, without manual labeling.

Decidability.. All five queries are decidable: they compose $(\cdot) \downarrow$ (computable by Theorem 2) with syntactic equality, finite set membership, or the skeleton map (a finite structural traversal), all of which are decidable. Richer queries combining structural predicates with the delta map δ (e.g., “all arbitrages with profit above threshold v through at least 3 pools”) remain decidable because δ is computed during normalization and the number of cycles in $T \downarrow$ is finite.

The rewriting system thus serves as the semantic foundation for a domain-specific language for fund flow analysis. The present paper instantiates one query (Q_{arb}); the remaining queries, and a systematic study of their expressiveness, are future work.